

7. Polymorphism

★ Polymorphism - (Poly - many morphism - forms)

When a particular method is defined in multiple formats having same name, same return type, but different arguments from such a multiple formats, only one format of method will be executed at a time. The execution depends on the method called situation, such a concepts are known as polymorphism.

In short - one entity - multiple formats is polymorphism.

★ Types of Polymorphism -

In Java, there are two types of Polymorphism -

I) Compile - time Polymorphism

II) Run - time Polymorphism

I) Compile time Polymorphism -

In a class when particular method is defined (constructed) is in more than one format having same name, same return type but different arguments from which only one format will be executed depends on the values passed from method calling section and such a mechanism handled by java compiler, is known as compiler time polymorphism.

Concept of compile time polymorphism is implemented through method overloading.

Compile time polymorphism is handled by java compiler.

Compile time polymorphism is also referred as Static polymorphism because at compile time, compiler knows that which format of method is going to be called depends on the passed values.

Compile time polymorphism is also referred as early binding (Static binding) because the association (interaction) between particular format of method and the method call section is maintained at compile time, before running the program.

In compile time polymorphism, multiple formats of method are defined in same class.

In compile time polymorphism, arguments of each format are different.

II) Runtime Polymorphism-

When a particular method is defined in more than one formats in super class or in sub class, from which only one format of method will be executed, at runtime and such a mechanism is handled by the JVM is known as runtime polymorphism.

Concepts of Runtime Polymorphism are implemented through method overriding.

Remember that-

In some cases, method overloading is also used to implement the concepts of Runtime Polymorphism.

Runtime polymorphism is handled by JVM.

Runtime polymorphism is also known as dynamic polymorphism because from multiple formats of method, which format of method is to call is decided by JVM at runtime. At runtime, depends on the inputs, JVM decides that, which format of method is to call, execute.

Runtime polymorphism is also referred as late binding (Dynamic binding) because the association (binding or interaction) between particular format of method and its calling statement is maintained at run time, after executing the program.

In runtime polymorphism - multiple formats of methods are defined in inheritance one format is defined in Super class where as other format is defined in Sub class.

In runtime polymorphism, arguments of all formats are same.

Concept of Compile time Polymorphism are implemented in programs through method overloading or through Constructor overloading.

Concept of runtime polymorphism are implemented through method overriding.

Remember that - Runtime polymorphism is also implemented using method overloading and Constructor overloading.

★ Method Overloading -

In a class, when particular method is defined in more than one formats where return type and method name of all formats is same but arguments / parameters of each format is different from all such formats of methods, Only one format of method will be executed at run time depends on types of values passed from method calling statement. Such a concept of implementation of methods is known as method overloading.

Generally, in a class, one method is constructed in only one format. When we construct the method in multiple formats having different arguments then that method becomes overload and Method overloading will raise.

Remember that -

"In method overloading, one method is defined into more than one formats when arguments / parameters of all formats are different from all such formats, only one format will be executed at run time.

Arguments of all formats are different in following 3 cases -

- 1) Number of arguments are different.
- 2) Sequence of arguments is different.
- 3) Data types of arguments are different.

We can also overload `main()` method. But JVM calls `main()` method which receives String array as arguments only.

Examples of method Overloading -

Following are various java programs that implements the concepts of method overloading indirectly implements the concepts of compile time polymorphism.

- 1) // java program using Method Overloading
- // Program to implement Compile time polymorphism
- // Program to find the addition and average.

```
class Addavg
```

```
{
    int add;
    double avg;
    void result (int a, int b)
    {
```

```
        System.out.println("Addition and average of 2
                             numbers" + a + " & " + b);
```

```
        add = a + b;
```

```
        avg = (double) add / 2;
```

```
    }
```

```
    void result (int x, int y, int z)
    {
```

```
        System.out.println("Addition & average of 3
```

```
        numbers: " + x + " & " + y + " & " +
        z);
```

```
        add = (double) x + y + z;
```

```
        avg = (double) add / 3;
```

```
    }
```

```
void result (int x, int y, int z, int w)
```

```
{
    System.out.println("Addition and Average of 4 numbers: "
        + x + " + " + y + " + " + z + " + " + w);
```

```
    add = x + y + z + w;
```

```
    avg = (double) add / 4;
```

```
void display ()
```

```
{
```

```
    System.out.println("Addition = " + add);
```

```
    System.out.println("Average = " + avg);
```

```
}
```

```
}
```

```
public class Method-Overloading-Demo
```

```
{
```

```
    public static void main (String args[])
```

```
{
```

```
        AddAddArg obj = new AddArg();
```

```
        obj.result (25, 40, 30); // Method Calling Statement
```

```
        obj.display ();
```

```
}
```

```
}
```

Output -

Addition and Average of 3 numbers 25 40 30

Addition = 95

Average = 31.6667

In main() method of above program, if we change second statement (method calling statement) in following format -

obj. result (20, 30);

then the output of above program will be displayed as -

output -

Addition and average of 2 numbers 20 30

Addition = 50

Average = 25

In above program in main() method, if we change second statement as -

obj. result (100, 200, 300, 400);

then the above program shows the output in following format -
output -

Addition and average of 4 numbers 100 200 300 400

Addition = 1000

Average = 250

In above program - from main() method, when calling result() method, if we pass 2 values then 1st format of result() method will execute, if we pass 3 values then second format of result() method will execute which have 3 arguments, if we pass 4 values then 3rd format of result() method will execute that have 4 arguments.

Note that- if we pass less values or greater values then java displays error at the time of compiling the program.

In above program- the method `result()` is constructed in 3 different formats from which at runtime, only one format will be executed all 3 formats performs 3 different tasks. So that, only one method `result()` can perform 3 tasks, so it is polymorphism.

This polymorphism is said as Compile time polymorphism because at the time of compiling the program, java compiler gets an idea that which format of method is going to call and execute.

★ Constructor Overloading -
Compile time polymorphism (Method overloading using Constructors) -

In a class, when more than one constructors are defined where constructor arguments / parameters are different from which only one constructor will be executed depends on the values passed from object creation statement. Such a concept are known as Constructor Overloading.

Example -

i) // Program using Constructor overloading.

// Program for compile time polymorphism using Constructor overloading.

```
class testing
```

```
{
```

```
    int add;
```

```
    double avg;
```

```
    testing ()
```

```
{
```

```
    System.out.println("First Constructor executes having  
no parameters");
```

```
    add = 0;
```

```
    avg = 0;
```

```
}
```

```
    testing (int a, int b)
```

```
{
```

```
    System.out.println("Second Constructor executes having  
2 parameters" + a + " + " + b);
```

```
    add = a + b;
```

```
    avg = (double) add / 2;
```

```
}
```

```
    testing (int a, int b, int c)
```

```
{
```

```
    System.out.println("Third Constructor executes having  
3 parameters" + a + " + " + b + " + " + c);
```

```
    add = a + b + c;
```

```
    avg = (double) add / 3;
```

```
}
```



```
void result()
{
    System.out.println("Addition = " + a + b);
    System.out.println("Average = " + avg);
}
```

```
public class Constructor_Overloading_Demo
{
    public static void main (String args[])
    {
        testing obj = new testing (20, 25);
        obj.result();
    }
}
```

In above program - at the time of creating the object, two values are passed so second constructor method is executed which have two parameters and the output of above program will be displayed as -

Second Constructor executes 2 parameters 20 25

Addition = 45

Average = 22.5

In above program, in main() method if we change the first statement in following format -

```
testing obj = new testing();
```

then the above program executes first constructor

because no parameters are passed & the output will be displayed as-

output -
First Constructor executes having no parameters.

Addition = 0

Average = 0.

In above program, if we change the 1st statement `main()` method in following format -

`testing obj = new testing (10, 20, 30);`

then third Constructor will be executed that have 3 parameters and the output of above program will be displayed as-

output -

Third Constructor executes having 3 parameters 10 20 30

Addition = 60

Average = 20.

2) Write a java program for runtime polymorphism using method overriding.

⇒

```
class inputs
```

```
{
```

```
double x;
```

```
void inputData()
```

```
{
```

```
    x = 7.5;
```

```
}
```



```
void display ()  
{  
    System.out.println("Radius of Circle = " + r);  
}
```

```
class Circle extends inputs
```

```
{  
    double a, c;
```

```
    void result ()  
    {
```

```
        inputData();
```

```
        a = 3.14 * r * r;
```

```
        c = 2 * 3.14 * r;
```

```
        Super.display();
```

```
        display();  
    }
```

```
    void display ()  
    {
```

```
        System.out.println("Area = " + a);
```

```
        System.out.println("Circumference = " + c);  
    }
```

```
}  
public static void main (String args[])
```

```
{  
    public class Method-Overriding-Demo
```

```
{  
    public static void main (String args[])
```

```
{  
        Circle ob = new Circle ();
```

```
        ob.result ();  
    }
```

```
}
```


★ Covariant Return Type-

The Covariant return type specifies that the return type may vary in the same direction as the subclass.

Before Java 5, it was not possible to override any method by changing the return type. But now, since Java 5, it is possible to override method by changing the return type.

Java doesn't allow the return type based overloading, but JVM always allows return type based overloading. JVM uses the full signature of a method for lookup/resolution. Full signature means it includes return type in addition to argument types - i.e. a class can have two or more methods differing only by return type. javac uses this fact to implement covariant return types.